

NLP

Module 1

-How does Google understand what we type?

Best pizza near me

Google understands:

- User wants restaurants
- User's location
- User's intent

-Translate "Good Morning" into Hindi

Output: सुप्रभात

How?

Because of **Natural Language Processing (NLP)**.

-Natural language is the language spoken and written by humans.

Examples

- English
- Hindi
- Gujarati
- French
- German
- Japanese

Example:

I love machine learning.

Humans easily understand this sentence.

Computers do **not** understand English directly.

Computers understand only

0 and 1

Therefore we need NLP.

-NLP:

Natural Language Processing (NLP) is a branch of Artificial Intelligence that enables computers to understand, interpret, analyze, and generate human language.

NLP is a field of Artificial Intelligence that allows computers to understand and process human language.

Artificial Intelligence (AI)



Machine Learning (ML)



Natural Language Processing (NLP)

Artificial Intelligence (AI)

AI is the ability of machines to perform tasks that normally require human intelligence.

Examples

- Chess
- Self-driving cars
- Face recognition
- Voice assistants

Machine Learning (ML)

Machine Learning is a subset of AI.

Instead of programming every rule, computers learn patterns from data.

Example

Spam email detection.

Natural Language Processing (NLP)

NLP is a specialized field that focuses on processing human language.

Examples

- Google Translate
- ChatGPT
- Grammarly
- Siri

Which of these are NLP applications?

- Calculator
- Google Translate
- ChatGPT
- Face Unlock
- Spell Checker

-Why Do We Need NLP?

Suppose a customer writes

This phone is fantastic!

Computer sees

"This"

"phone"

"is"

"fantastic"

Computer doesn't know

Fantastic = Positive

NLP teaches computers

Fantastic → Positive Emotion

Another example

I am not happy.

NLP understands

Sentiment = Negative

-Difference

Natural Language	Programming Language
-------------------------	-----------------------------

Used by humans	Used by computers
----------------	-------------------

Ambiguous	Precise
-----------	---------

Grammar varies	Strict syntax
----------------	---------------

Multiple meanings	One meaning
-------------------	-------------

Example: English	Example: Python
------------------	-----------------

Example

Natural Language

Open the window.

Meaning depends on context.

Programming

```
print("Hello")
```

- Components of NLP

Two major components:

1. Natural Language Understanding (NLU)

Computer understands text.

Examples

- Intent detection
- Question answering
- Sentiment analysis

Example

I am very happy today.

NLU determines

Positive sentiment

2. Natural Language Generation (NLG)

Computer generates text.

Examples

- ChatGPT
- AI writing assistants
- Email generation

Example

Input

Summarize this article.

Output

Generated summary.

-Applications of NLP

1. Chatbots

Example

- ChatGPT
- Customer Support Bots

Used in

- Banks
- Airlines
- E-commerce

2. Machine Translation

Example

English

How are you?

Hindi

आप कैसे हैं?

3. Sentiment Analysis

Input

This laptop is amazing.

Output

Positive

Input

Worst phone ever.

Output

Negative

4. Spam Detection

Email

Congratulations!
You won ₹10,00,000.

Spam

5. Voice Assistant

Examples

- Siri
- Alexa
- Google Assistant

User says

Play music.

Assistant understands command.

6. Search Engines

User searches

Best engineering colleges in Gujarat

Google understands

- Best
- Engineering colleges
- Gujarat

Returns relevant results.

7. Grammar Checker

Example

Input

He go to school.

Corrected

He goes to school.

8. Text Summarization

Long article

↓

Short summary

9. Question Answering Systems

Example

ChatGPT

Google Gemini

Microsoft Copilot

10. Recommendation Systems

Netflix

Amazon

YouTube

Recommend content using text and user interactions.

- Challenges in NLP

Ambiguity

Example

I saw a man with a telescope.

Who has the telescope?

Multiple Languages

India has many languages.

English

Hindi

Gujarati

Marathi

Tamil

NLP must support all.

Spelling Errors

Example

Gud Morning

Should be

Good Morning

Slang

Example

LOL

OMG

BRB

Need interpretation.

Text preprocessing is the process of cleaning and transforming raw text into a structured format before applying NLP or Machine Learning algorithms.

Raw Text

"Machine Learning is AMAZING!!! It helps computers in 2026."

After preprocessing

machine learning amazing helps computers

Notice that:

- Converted to lowercase
- Removed punctuation
- Removed numbers
- Removed stop words

Text Preprocessing Pipeline

Raw Text

|



Lowercase Conversion

|



Tokenization

|



Remove Punctuation

|



Remove Stop Words

|



Stemming / Lemmatization

|



Clean Text

1. Lowercase Conversion

Definition

Converts all characters to lowercase so that words with different capitalization are treated as the same word.

Example

Before

Machine Learning NLP

After

machine learning nlp

Python

```
text = "Machine Learning IS Amazing!"  
print(text.lower())
```

Output

```
machine learning is amazing!
```

2. Tokenization

Tokenization is the process of dividing text into smaller units called tokens.

Tokens may be:

- Words
 - Sentences
 - Characters
-

Word Tokenization

Sentence

```
Machine learning is amazing
```

```
["Machine", "learning", "is", "amazing"]
```

Sentence Tokenization

Paragraph

```
NLP is interesting. It is used in chatbots.
```

↓

```
Sentence 1
```

```
Sentence 2
```

NLTK Program:

```
from nltk.tokenize import word_tokenize
```

```
text = "Machine learning is amazing."
```

```
tokens = word_tokenize(text)
```

```
print(tokens)
```

```
O/P:['Machine', 'learning', 'is', 'amazing', '.']
```

3. Remove Punctuation

Punctuation marks generally do not contribute to the meaning required for many NLP tasks.

Examples:

- .
- ,
- !
- ?
- :
- ;

Hello!!!

After

Hello

Python

```
import string
tokens = [word for word in tokens if word not in string.punctuation]
```

4. Stop Word Removal

Stop words are common words that usually carry little meaning.

Example: Stop words are common words that usually carry little meaning.

is

the

a

an

of

to

in

and

Python:

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))
```

```
filtered = [word for word in tokens if word.lower() not in stop_words]
```

5. Stemming

Stemming removes prefixes or suffixes to obtain the **root form** of a word.

The result **may not be a valid dictionary word**.

Examples

Original	Stemmed
----------	---------

playing	play
---------	------

running	run
---------	-----

studies	studi
---------	-------

happiness	happi
-----------	-------

Python:

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
print(stemmer.stem("running"))
```

6. Lemmatization

Lemmatization converts words to their **dictionary (base) form**, considering grammar and vocabulary.

```
from nltk.stem import WordNetLemmatizer
```

```
lemmatizer = WordNetLemmatizer()
```

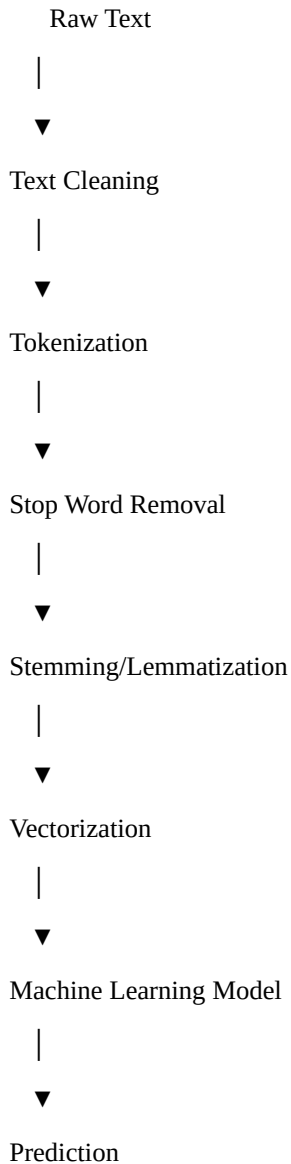
```
print(lemmatizer.lemmatize("children"))
```

O/P: child

Library	Purpose
NLTK	Educational NLP library with tokenization, stop-word removal, stemming, and lemmatization
spaCy	Industrial NLP library with fast tokenization, lemmatization, POS tagging, and NER
TextBlob	Simple NLP library for tokenization, sentiment analysis, spell correction, and noun phrase extraction

A **pipeline** is a sequence of steps performed to convert **raw text** into a form that a computer can analyze.

An **NLP pipeline** is a series of preprocessing and analysis steps that transform raw text into structured numerical data for machine learning or analytics.



Example

Raw text:

"I absolutely love this mobile phone!"

Computer cannot understand this sentence directly.

It first converts it through the NLP pipeline.

*** Stages of NLP Analytics**

a) Data Collection

Sources:

- Emails
- Twitter/X posts
- News articles
- Reviews
- Chat messages

b) Text Preprocessing

- Lowercase
- Remove punctuation
- Remove numbers
- Remove stop words
- Stemming/Lemmatization

c) Tokenization

Machine learning is interesting.

Into

["Machine", "learning", "is", "interesting"]

d) Feature Extraction (Vectorization)

I love NLP

into

[0, 2, 1, 0, 3]

*** Bag of Words**

BoW converts text into numerical vectors by **counting the frequency of each word**.

Document:

D1: I love NLP

D2: NLP is interesting

Vocabulary:

love

nlp

interesting

is

BoW Matrix:

Document	love	nlp	interesting	is
D1	1	1	0	0
D2	0	1	1	1

* TF-IDF

TF-IDF assigns weights to words.

Rare words receive higher importance.

Common words receive lower importance.

Formula:

$$TFIDF = TF \times IDF$$

Documents

D1: cat sat mat

D2: cat ate fish

Word "cat"

Appears everywhere

↓

Low IDF

Word "fish"

Appears once

↓

High IDF

Unique words are:

Word

i

love

nlp

is

interesting

Total documents:

N=2

Calculate Document Frequency (DF)

Word	Appears In	DF
i	D1	1
love	D1	1
nlp	D1, D2	2
is	D2	1
interesting	D2	1

Calculate IDF

Using the simple formula:

$$\text{IDF} = \log(\text{DFN})$$

where $N=2$.

Word	DF	$\text{IDF} = \log_e(2/\text{DF})$
i	1	0.693
love	1	0.693
nlp	2	0.000
is	1	0.693
interesting	1	0.693

Calculate TF (Term Frequency)

Document 1

I love NLP

Total words = 3

Word	Count	TF
i	1	$1/3 = 0.333$
love	1	0.333
nlp	1	0.333

Document 2

NLP is interesting

Total words = 3

Word	Count	TF
nlp	1	0.333
is	1	0.333
interesting	1	0.333

Calculate TF-IDF

$$TFIDF = TF \times IDF$$

Document 1

Word	TF	IDF	TF-IDF
i	0.333	0.693	0.231
love	0.333	0.693	0.231
nlp	0.333	0.000	0.000
is	0	0.693	0
interesting	0	0.693	0

Document 2

Word	TF	IDF	TF-IDF
i	0	0.693	0
love	0	0.693	0
nlp	0.333	0.000	0.000
is	0.333	0.693	0.231
interesting	0.333	0.693	0.231

Final TF-IDF Matrix

Document	I	Love	NLP	Is	Interesting
D1	0.231	0.231	0.000	0.000	0.000
D2	0.000	0.000	0.000	0.231	0.231

BoW	TF-IDF
Frequency	Weight
Easy	Better
Common words dominate	Rare words dominate

Entropy measures uncertainty or randomness.

Higher uncertainty

→ Higher entropy

Lower uncertainty

→ Lower entropy

Example 1

Tomorrow's weather

Sunny

Rainy

Cloudy

Storm

Very difficult to predict.

Entropy is high.

Example 2

Traffic signal

Red

Green

Yellow

Fairly predictable.

Entropy is lower.

Example 3

Sun rises every morning.

100% sure.

Entropy = 0

Formula

$$H(X) = -\sum p(x) \log_2 p(x)$$

Where

- $H(X)$ = Entropy
- $p(x)$ = Probability of an event
- $\log_2$ = Base-2 logarithm

Example 1

Fair Coin

Head=0.5

Tail=0.5

Entropy

$$H=-(0.5\log_2 0.5 + 0.5\log_2 0.5)$$

Since

$$\log_2(0.5)=-1$$

$$H=-(0.5(-1) + 0.5(-1))$$

$$H=1$$

Maximum uncertainty.

$$=1$$

Maximum uncertainty.

Example 2

Always Head

$$\text{Head}=1$$

$$\text{Tail}=0$$

Entropy

$$=0$$

No uncertainty.

$$H=-(1 \times \log_2 1)$$

Since

$$\log_2 1=0$$

Entropy

$$H=0$$

Document

-apple apple apple apple

Low entropy

-apple banana mango orange

High entropy

Entropy in NLP

Sentence 1

apple apple apple apple

Probability

Apple = 1

Entropy

0

Reason

Every word is the same.

Sentence 2

apple banana mango orange

Probability

Each word

0.25

Entropy

High

Reason

Many different words appear.

Higher randomness.

Applications of Entropy

- Decision Trees
- Text Compression
- Language Modeling
- NLP
- Data Mining
- Information Theory

- Cross Entropy

Measures how close predicted probabilities are to actual probabilities.

Formula

$$H(P,Q) = -\sum P(x) \log Q(x)$$

Example

Actual

Spam=1

Prediction

Spam=0.95

↓

Small Cross Entropy

Prediction

Spam=0.20

↓

Large Cross Entropy

Cross entropy measures how different the predicted probability distribution is from the actual probability distribution.

Lower value-Better model.

Higher value-Poor model.

Entropy

Measures uncertainty

One distribution

Information Theory

Cross Entropy

Measures prediction error

Actual vs Predicted

ML Loss Function

Formula

$$H(P,Q) = -\sum P(x) \log(Q(x))$$

Where

P

Actual probability

Q

Predicted probability

Example

Suppose

Actual

Spam = 1

Not Spam = 0

Prediction

Spam = 0.90

Not Spam = 0.10

Cross entropy

$-(1 \times \log(0.9))$

Very small.

NLP Example

Sentence

I love NLP

Model predicts

Positive = 0.98

Actual

Positive = 1

Cross entropy is very small.

Wrong prediction

Positive = 0.05

Cross entropy becomes very large.

Applications:

Text Classification

- Sentiment Analysis
- Spam Detection

- Machine Translation
- BERT
- GPT
- Chatbots

Text Encoding & Word Embeddings

1. Character Encoding

Character Encoding converts characters into numbers.

ASCII

Full Form

American Standard Code for Information Interchange

CAT

The computer stores it as numbers.

C → 67

A → 65

T → 84

Internally these numbers are stored in binary.

67 = 1000011

65 = 1000001

84 = 1010100

```
print(chr(65))  
print(chr(66))  
print(chr(97))
```

Output

A
B
a

```
print(ord('A'))  
print(ord('B'))  
print(ord('a'))
```

Output

65
66
97

Limitations of ASCII

ASCII supports only English characters.

It cannot represent:

- Hindi
- Gujarati
- Chinese
- Arabic
- Emojis

Example

नमस्ते

ગુજરાતી

ASCII cannot encode these characters

Unicode

Unicode was developed to overcome the limitations of ASCII.

It assigns a unique code to every character from almost every language.

Examples

Character	Unicode
A	U+0041
₹	U+20B9
	U+1F60A
न	U+0928

UTF-8

UTF-8 stands for **Unicode Transformation Format – 8-bit**.

It is the most widely used encoding format.

Features:

- Compatible with ASCII.
- Variable length (1–4 bytes per character).
- Efficient storage.
- Standard encoding for web pages and NLP datasets.

Character Encoding in NLP

Before applying NLP techniques, text must be encoded correctly.

Example

Dr. xyz teaches NLP.

↓

Characters

↓

Unicode values

↓

Computer processing

↓

NLP tasks

Word Embeddings

Word embeddings represent words using dense numerical vectors.

Example

King $\rightarrow$ [0.42, 0.81, -0.13...]

Queen $\rightarrow$ [0.45, 0.78, -0.10...]

Since the vectors are similar, the model learns that **King** and **Queen** are semantically related.

Common embedding techniques:

- Word2Vec
- GloVe
- FastText
- BERT Embeddings

One-Hot Encoding

Suppose the vocabulary is:

Apple

Banana

Mango

Orange

Word Vector

Apple [1 0 0 0]

Banana [0 1 0 0]

Mango [0 0 1 0]

Orange [0 0 0 1]

NLP Tools & Frameworks

1. NLTK

Natural Language Toolkit

Features

- Tokenization
- Stop Words
- Stemming
- Lemmatization
- POS Tagging
- Basic NER

Best for

Teaching and research.

2. spaCy

Features

- Fast processing
- Tokenization
- Lemmatization
- POS Tagging
- Named Entity Recognition
- Dependency Parsing

Best for

Industry applications.

3. TextBlob

Features

- Tokenization
- Sentiment Analysis
- Spell Correction
- Noun Phrase Extraction

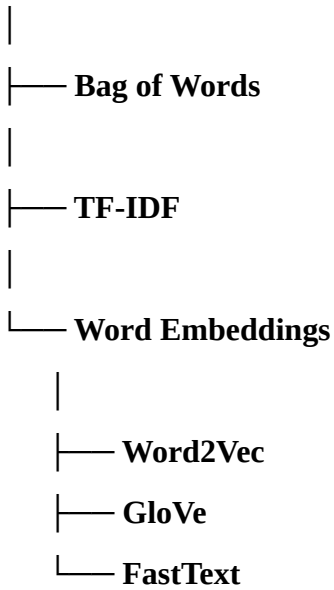
Best for

Beginners and quick NLP tasks.

MODULE 2

Text Representation (Text Vectorization)

Text



-Bag of Words (BoW)

Definition

Bag of Words is the simplest text representation technique.

It counts the frequency of each word in a document.

Word order is ignored.

Example:

D1 : I love NLP

D2 : NLP is interesting

D3 : I love Machine Learning

Step 1:

love

nlp

is

interesting

machine

learning

Step 2:

Count each word

Document	love	nlp	is	interesting	machine	learning
----------	------	-----	----	-------------	---------	----------

D1	1	1	0	0	0	0
----	---	---	---	---	---	---

D2	0	1	1	1	0	0
----	---	---	---	---	---	---

D3	1	0	0	0	1	1
----	---	---	---	---	---	---

Code:

```
From sklearn.feature_extraction.text import CountVectorizer
```

```
documents = [
```

```
- "I love machine learning",  
  "Machine learning is very interesting",  
  "I love NLP and machine learning",  
  "NLP is used in artificial intelligence"
```

```
]
```

```
vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(documents)
```

```
print(vectorizer.get_feature_names_out())
```

```
print(X.toarray())
```

```
*****
```

Introduction to Text Representation and Bag of Words

Text Representation?

- Computers understand numbers, not text.

- Text must be converted into numerical vectors.
- This process is called **Text Vectorization**.

Example:

Sentence:

I love NLP

↓

Computer

↓

[0,1,0,1,2]

Applications

- Spam Detection
- Sentiment Analysis
- Machine Translation
- Chatbots
- Text Classification

Types of Text Representation

1. Bag of Words (BoW)
2. TF-IDF
3. Word2Vec
4. GloVe
5. FastText
6. BERT Embeddings

BoW:

Bag of Words is one of the simplest text representation techniques. It represents a document by counting the frequency of each word, without considering the order of words.

Advantages of Bag of Words

- Simple to understand and implement.
- Fast computation.
- Works well for small datasets.
- Effective for basic text classification tasks.

Limitations of Bag of Words

- Ignores word order.
- Ignores context and meaning.

- Produces sparse matrices for large vocabularies.
- Cannot identify synonyms (e.g., *car* and *automobile* are treated as different words).

Code:

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd
```

```
# Sample documents
```

```
documents = [
    "I love machine learning",
    "Machine learning is interesting",
    "I love NLP",
    "NLP uses machine learning"
]
```

```
# Create CountVectorizer object
```

```
vectorizer = CountVectorizer()
```

```
# Convert text into BoW matrix
```

```
X = vectorizer.fit_transform(documents)
```

```
# Display vocabulary
```

```
print("Vocabulary:")
print(vectorizer.get_feature_names_out())
```

```
# Convert matrix to DataFrame
```

```
bow_df = pd.DataFrame(
    X.toarray(),
    columns=vectorizer.get_feature_names_out()
)
```

```
print("\nBag of Words Matrix:")
```

```
print(bow_df)
```

```
from
```

```
vectorizer.get_feature_names_out()
```

Returns all unique words (the vocabulary).

```
X.toarray()
```

Converts the sparse matrix into a regular array for easier viewing.

Real-World Applications

- Spam email detection
- Fake news classification
- Document categorization
- Topic identification
- Customer review analysis

TFIDF-

TF-IDF (Term Frequency–Inverse Document Frequency) is a statistical technique that measures the importance of a word in a document relative to a collection of documents.

Words that appear frequently in a document but rarely across other documents receive higher TF-IDF scores.

Components of TF-IDF

A. Term Frequency (TF)

Term Frequency measures how often a word appears in a document.

Formula

$$TF = \frac{\text{Number of times the word appears in the document}}{\text{Total number of words in the document}}$$

Example

Document:

I love NLP NLP

Word Frequency

I 1

love 1

NLP 2

Total words = 4

TF values:

Word	TF
I	$1/4 = 0.25$
love	$1/4 = 0.25$
NLP	$2/4 = 0.50$

B. Document Frequency (DF)

Document Frequency is the number of documents that contain a particular word.

Example:

D1: I love NLP

D2: NLP is interesting

D3: Machine learning uses NLP

Word	DF
NLP	3
love	1
Machine	1
interesting	1

C. Inverse Document Frequency (IDF)

IDF reduces the importance of words that appear in many documents.

Formula

$$\text{IDF} = \log(\text{DFN})$$

where:

- **N** = Total number of documents
 - **DF** = Number of documents containing the word
-

Example

Suppose:

- Total documents = 3

Word **NLP**

$$\text{DF} = 3$$

$$\text{IDF} = \log(3/3) = \log(1) = 0$$

Word **Machine**

$$\text{DF} = 1$$

$$\text{IDF} = \log(3/1) = 0.4771$$

This means **Machine** is more informative than **NLP** in this small collection.

3. TF-IDF Formula

$TF-IDF = TF \times IDF$

A word receives a **high TF-IDF score** if:

- It appears frequently in one document.
- It appears rarely in other documents.

4. Manual Example

Consider the following documents:

D1 : I love NLP

D2 : NLP is interesting

Step 1: Vocabulary

I
love
NLP
is
interesting

Step 2: Calculate TF

D1

Total words = 3

Word	TF
I	$1/3 = 0.333$
love	$1/3 = 0.333$
NLP	$1/3 = 0.333$

D2

Word	TF
NLP	$1/3 = 0.333$
is	$1/3 = 0.333$
interesting	$1/3 = 0.333$

Step 3: Calculate DF

Word	DF
I	1
love	1
NLP	2
is	1
interesting	1

Total Documents = 2

Step 4: Calculate IDF

Word	IDF
I	$\log(2/1)=0.301$
love	0.301
NLP	$\log(2/2)=0$
is	0.301
interesting	0.301

Step 5: Calculate TF-IDF

For **love** in D1:

$$0.333 \times 0.301 = 0.100$$

For **NLP**:

$$0.333 \times 0 = 0$$

Thus, **love** receives a higher weight than **NLP**, because NLP appears in every document.

5. Python Implementation

Syntax

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

Program

```
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

documents = [
    "I love machine learning",
    "Machine learning is interesting",
    "I love NLP",
    "NLP uses machine learning"
]

# Create TF-IDF object
vectorizer = TfidfVectorizer()

# Convert documents
X = vectorizer.fit_transform(documents)

# Vocabulary
print("Vocabulary:")
print(vectorizer.get_feature_names_out())

# Convert matrix to DataFrame
tfidf_df = pd.DataFrame(
    X.toarray(),
    columns=vectorizer.get_feature_names_out()
)
```

```
print("\nTF-IDF Matrix:")
print(tfidf_df.round(3))
```

Expected Output (Example)

Vocabulary

```
['interesting', 'is', 'learning', 'love', 'machine', 'nlp', 'uses']
```

TF-IDF Matrix

Document	interesting	is	learning	love	machine	nlp	uses
D1	0.000	0.000	0.533	0.658	0.533	0.000	0.000
D2	0.539	0.425	0.344	0.000	0.344	0.000	0.000
D3	0.000	0.000	0.000	0.451	0.000	0.451	0.000
D4	0.000	0.000	0.436	0.000	0.436	0.344	0.540

Code Explanation

```
vectorizer = TfidfVectorizer()
```

Creates a TF-IDF vectorizer object.

```
X = vectorizer.fit_transform(documents)
```

Builds the vocabulary and computes TF-IDF scores.

```
vectorizer.get_feature_names_out()
```

Displays all unique words in the vocabulary.

```
X.toarray()
```

Converts the sparse TF-IDF matrix into a dense array for display.

```
*****
```

Word2Vec?

Word2Vec is a neural network-based algorithm developed by **Google** that learns word meanings from their surrounding words (context).

Main Idea

Words that appear in similar contexts have similar meanings.

Example

Sentence:

The cat drinks milk.
The dog drinks milk.

The words **cat** and **dog** appear in similar contexts:

_____ drinks milk

Hence, Word2Vec learns that **cat** and **dog** are related.

Two Architectures

1. CBOW (Continuous Bag of Words)

Predict the **missing word** from surrounding words.

Example:

I _____ NLP

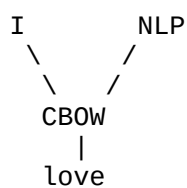
Context:

I
NLP

Prediction:

love

Diagram:



2. Skip-Gram

Predict surrounding words using the center word.

Example:

I love NLP

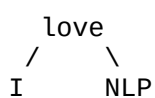
Input:

love

Output:

I
NLP

Diagram:



Advantages

- Captures semantic meaning.
- Fast to train.
- Widely used.

Disadvantages

- Cannot handle unknown words.
- One vector per word.

Python Example

```
from gensim.models import Word2Vec

sentences = [
    ["i", "love", "nlp"],
    ["i", "love", "machine", "learning"],
    ["nlp", "is", "interesting"]
]

model = Word2Vec(sentences, vector_size=50, window=2, min_count=1)

print(model.wv["love"])
```

Similar Words

```
print(model.wv.most_similar("love"))
```

Example Output:

```
nlp
machine
learning
```

3. GloVe

Full Form

Global Vectors for Word Representation

Developed by **Stanford University**.

Unlike Word2Vec, GloVe uses **global word co-occurrence statistics** from the entire corpus.

Simple Example

Corpus:

```
I love NLP.
I love AI.
AI loves data.
```

Word Co-occurrence Table

Word ↓ / Context →	I	Love	NLP	AI
I	0	2	0	0
Love	2	0	1	1
NLP	0	1	0	0
AI	0	1	0	0

GloVe uses this table to learn vectors.

Advantages

- Uses global information.
- Better semantic relationships.
- Good for large datasets.

Disadvantages

- Needs a large corpus.
- Pre-training can be expensive.

Example Relationships

King → Queen

Man → Woman

Paris → France

Delhi → India

4. FastText

Developed by **Facebook (Meta)**.

FastText represents a word using **subwords (character n-grams)** instead of treating the whole word as a single unit.

Example

Word:

playing

Subwords (3-grams for illustration):

pla
lay
ayi
yin
ing

FastText learns vectors for these subwords and combines them.

playings

FastText:

Recognizes subwords:

play
ing

So it can still generate a meaningful vector.

Advantages

- Handles unknown words.
- Works well with spelling mistakes.
- Better for morphologically rich languages.

Python Example

```
from gensim.models import FastText

sentences = [
    ["machine", "learning"],
    ["deep", "learning"],
    ["artificial", "intelligence"]
]

model = FastText(sentences, vector_size=50, window=2, min_count=1)

print(model.wv["learning"])
```

Comparison

Feature	Word2Vec	GloVe	FastText
Developed By	Google	Stanford	Facebook (Meta)
Uses Context	Yes	Yes	Yes
Uses Global Statistics	No	Yes	No
Uses Character N-grams	No	No	Yes
Handles Unknown Words	No	No	Yes
Training Speed	Fast	Moderate	Fast
Best For	General NLP	Semantic Analysis	Rich Morphology & Misspellings

Real-Life Applications

Word2Vec

- Document similarity
- Recommendation systems
- Chatbots

GloVe

- Machine Translation
 - Search Engines
 - Question Answering
-

FastText

- Spell Checking
 - Language Identification
 - Social Media Text Analysis
 - Morphologically rich languages
-

Try it:

Consider the following words:

Cat
Dog
Tiger
Car
Bus

Questions

1. Which two words are most similar?
 2. Which words belong to the same category?
 3. Which algorithm can recognize the word "**cats**" even if it was not seen during training?
-

Expected Answers

1. **Cat** and **Dog**
 2. **Cat, Dog, Tiger**
 3. **FastText**
-

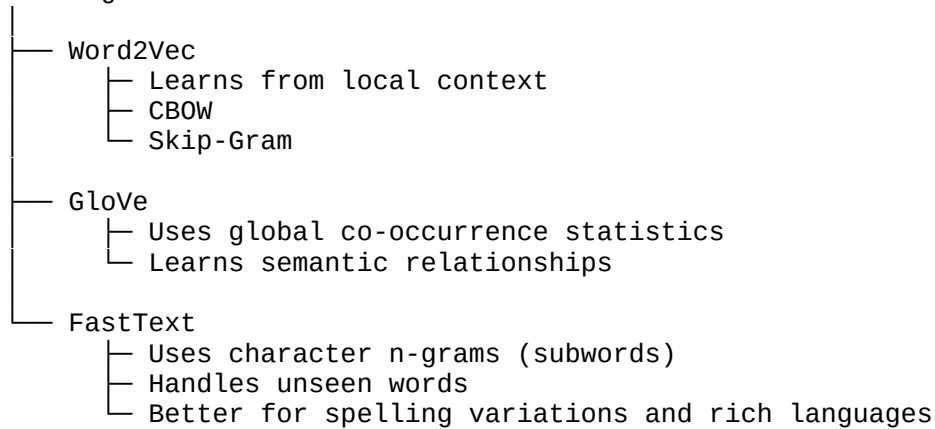
Sample Questions

1. What is a word embedding?
2. Why are word embeddings better than one-hot encoding?
3. What is the main idea of Word2Vec?
4. What is the difference between CBOW and Skip-Gram?
5. What is GloVe?
6. What does FastText use to represent words?
7. Which algorithm handles unseen words better?
8. Which algorithm uses global co-occurrence statistics?

9. Name one application of Word2Vec.
10. Name one advantage of FastText.

Summary

Word Embeddings



Word Embedding

An embedding is a numerical representation of a word in a continuous vector space.

Example

Instead of

Python

BoW

0 0 1 0 0 0

Word2Vec

[0.23, -0.61, 0.91, 0.12, ...]

Usually

- 50 dimensions
- 100 dimensions
- 200 dimensions
- 300 dimensions

Step 1: Our Dataset

Sentence 1:

I love machine learning

Sentence 2:

Machine learning is interesting

Step 2: Vocabulary

Collect unique words.

Index	Word
1	I
2	love
3	machine
4	learning
5	is
6	interesting

Initially, the computer knows **nothing** about these words.

Step 3: Random Numbers

Word2Vec assigns random vectors.

Word	Vector
I	[0.12,0.54]
love	[-0.2,0.6]
machine	[0.9,-0.4]
learning	[-0.8,0.3]

These numbers are completely random.

Step 4: Context Window

Suppose window size = 1.

Sentence

I love machine learning

For the word **love**, the neighbouring words are:

I	love	machine
↑		↑
Context		Context

Target Word = **love**

Context Words = **I, machine**

Another example:

Target

machine

Context

love

learning

Step 5: Training

The neural network sees

Input

I

machine

Correct Answer

love

Initially, it predicts the wrong word.

Example

Predicted

interesting

Actual

love

Wrong prediction!

So the neural network adjusts the vectors slightly.

Step 6: Repeat Thousands of Times

Again

machine learning

Again

machine learning

Again

machine learning

Again

machine learning

After seeing these words together many times, the vectors become similar.

Example

Before Training

Word	Vector
------	--------

machine	[0.9,-0.3]
---------	------------

Word	Vector
learning	[-0.5,0.8]

learning [-0.5,0.8]

Very different.

****After Training**

Word	Vector
machine	[0.61,0.83]
learning	[0.59,0.81]

machine [0.61,0.83]

learning [0.59,0.81]

Now they are almost the same.

Because they appeared together many times.

Word2Vec thinks

If two words appear in similar contexts, they probably have similar meanings.

Difference from TF-IDF

TF-IDF asks

How many times did the word occur?

Example

Machine = 5

Learning = 6

That's all.

Word2Vec asks

Which words usually appear together?

Example

Machine

Learning

Artificial Intelligence

Algorithm

Since they often appear in the same context, Word2Vec places their vectors close together.

Easy Analogy

Imagine a classroom.

Students who always sit together become friends.

Similarly,

Words that always occur together become **neighbours in vector space**.

Machine Learning

AI

Algorithm

Far away

Banana

Apple

Orange

So "Machine" is close to "Learning" but far from "Banana".

The Formula

Unlike TF-IDF, **there is no direct formula** such as:

$TF = \text{count} / \text{total words}$

Word2Vec is trained using a neural network that:

1. Starts with **random vectors**.
2. Creates **context–target pairs**.
3. Predicts the target from the context (CBOW) or the context from the target.
4. Calculates the prediction error (loss).
5. Updates the vectors using **backpropagation**.
6. Repeats this process many times until similar-context words have similar vectors.

-
- **Bag of Words** → Counts words.
 - **TF-IDF** → Weighs words.
 - **Word2Vec** → Learns the meaning of words from their context.

Case 2: Window Size = 2

Now Word2Vec looks **2 words on each side**.

Sentence

I love machine learning

Target = **machine**

Context

I
love
learning

Since there are only three surrounding words, all are used.

CBOW

Input:
I
love
learning

Output:
machine

Feature Extraction?

Definition

Feature extraction is the process of converting raw text into useful features that machine learning models can understand.

Examples of NLP features:

- N-Grams
- Part-of-Speech (POS) Tags
- Named Entity Recognition (NER)
- TF-IDF Features
- Word Embeddings (Word2Vec)

N-Grams:

An **N-Gram** is a sequence of **N consecutive words** from a sentence.

Where:

- **N = 1** → Unigram
- **N = 2** → Bigram
- **N = 3** → Trigram
- **N = 4** → Four-gram

sentence:

I love machine learning

Unigram (N = 1)

Single words.

(I)
(love)
(machine)
(learning)

Bigram (N = 2)

Two consecutive words.

(I love)
(love machine)
(machine learning)

Trigram (N = 3)

Three consecutive words.

(I love machine)
(love machine learning)

Why are N-Grams Important?

Suppose we have:

New York

BoW treats

New

York

as separate words.

Bigram keeps them together.

(New York)

This preserves more context.

Applications of N-Grams

- Auto-complete
- Spell Checker
- Machine Translation
- Speech Recognition
- Search Engines

- Text Prediction
- Chatbots

Sentence:

I love NLP

Total words = 3

Unigrams

I

love

NLP

Count

Word	Count
I	1
love	1
NLP	1

Unigram Probability

Formula

$$P(\text{word}) = \frac{\text{Count}(\text{word})}{\text{Total words}}$$

For I

$$P(I) = \frac{1}{3} = 0.333$$

Similarly

$$P(\text{love}) = 0.333 \quad P(\text{NLP}) = 0.333$$

Bigram

Sentence

I love NLP

Bigrams

(I love)

(love NLP)

Total Bigrams = 2

Count

Bigram Count

I love 1

love NLP 1

Bigram Probability

Formula

$P(w_2 | w_1) = \frac{\text{Count}(w_1, w_2)}{\text{Count}(w_1)}$

Example

$P(\text{love} | \text{I})$

$\text{Count}(\text{I love}) = 1$

$\text{Count}(\text{I}) = 1$

$P = 1/1 = 1$

Trigram

Sentence

I love machine learning

Trigrams

(I love machine)

(love machine learning)

Probability Formula

$P(w_3 | w_1, w_2) = \frac{\text{Count}(w_1, w_2, w_3)}{\text{Count}(w_1, w_2)}$

Example

$P(\text{machine} | \text{I love})$

$\text{Count}(\text{I love machine}) = 1$

$\text{Count}(\text{I love}) = 1$

Probability

$1/1 = 1$

4. Python Implementation

Install NLTK

```
pip install nltk
```

Import Libraries

```
import nltk
from nltk.util import ngrams
from collections import Counter
```

Program

```
import nltk
from nltk.util import ngrams
from collections import Counter

text = "I love NLP I love machine learning"

# Convert to lowercase
tokens = text.lower().split()

# Generate N-grams
unigrams = list(ngrams(tokens,1))
bigrams = list(ngrams(tokens,2))
trigrams = list(ngrams(tokens,3))

print("Unigrams")
print(Counter(unigrams))

print("\nBigrams")
print(Counter(bigrams))

print("\nTrigrams")
print(Counter(trigrams))
```

Output

Unigrams

```
Counter({
('i',):2,
('love',):2,
('nlp',):1,
('machine',):1,
('learning',):1
})
```

Bigrams

```
Counter({
('i','love'):2,
('love','nlp'):1,
('nlp','i'):1,
('love','machine'):1,
('machine','learning'):1
})
```

Trigrams

```
Counter({
('i', 'love', 'nlp'):1,
('love', 'nlp', 'i'):1,
('nlp', 'i', 'love'):1,
('i', 'love', 'machine'):1,
('love', 'machine', 'learning'):1
})
```

Calculate Unigram Probability

```
from collections import Counter

counts = Counter(tokens)

total_words = len(tokens)

print("Unigram Probability\n")

for word, count in counts.items():
    probability = count / total_words
    print(word, "=", round(probability,3))
```

Output

i = 0.286

love = 0.286

nlp = 0.143

machine = 0.143

learning = 0.143

Sentence:

I love NLP I love machine learning

Tokens:

I
love
NLP
I
love
machine
learning

Total words = 7

Count of **I** = 2

$P(I) = \frac{2}{7} = 0.286$

Similarly:

- **love** = $\frac{2}{7} = 0.286$

- **NLP** = $1/7 = 0.143$
- **machine** = $1/7 = 0.143$
- **learning** = $1/7 = 0.143$

Advantages of N-Grams

- Simple to implement.
- Preserves local context.
- Improves language modeling.
- Useful for predictive text and search.

Limitations of N-Grams

- Vocabulary grows rapidly for larger values of N.
- Sparse data problem.
- Cannot capture long-range dependencies.
- Higher memory requirements.

Task:

- Unigrams
- Bigrams
- Trigrams

for the sentence:

Artificial Intelligence improves healthcare rapidly

Then:

1. Count each N-gram.
2. Calculate unigram probabilities.
3. Identify the most frequent unigram.

$P(\text{word}) = \frac{\text{Total WordsCount}(\text{word})}{\text{Total words}}$

Total words = 5

Word	Count	Probability
artificial	1	$1/5 = 0.20$
intelligence	1	$1/5 = 0.20$
improves	1	$1/5 = 0.20$
healthcare	1	$1/5 = 0.20$
rapidly	1	$1/5 = 0.20$

Output

Unigram Probability

artificial = 0.20
intelligence = 0.20
improves = 0.20
healthcare = 0.20
rapidly = 0.20

5. Most Frequent Unigram

Since **every word appears exactly once**, all words have the same highest frequency.

Word	Frequency
------	-----------

artificial	1
------------	---

intelligence	1
--------------	---

improves	1
----------	---

healthcare	1
------------	---

rapidly	1
---------	---

Answer: There is **no single most frequent unigram**. All five words are equally frequent (frequency =1)

Dimensionality Reduction: Techniques like PCA and LDA for reducing feature space in text analytics.

A dimension is a feature or attribute used to describe data.

Example 1: Student Data

Student	Age	CGPA	Attendance
A	20	8.5	90
B	21	7.9	85

Here, there are 3 features (dimensions):

- Age
- CGPA
- Attendance

Example 2: Text Data

Sentence:

Machine learning is interesting.

Vocabulary:

Machine
Learning
Interesting
Artificial
Data
Python
Computer

...

Each unique word becomes one feature.

If there are **10,000 unique words**, then each document has **10,000 dimensions**.

Problems of High Dimensionality

1. More Memory Required

Large vectors consume more RAM.

2. Slower Computation

Training machine learning models takes longer.

3. Noise

Some words contribute very little to prediction.

Examples:

the
is
of
and

These words add dimensions but little useful information.

4. Overfitting

Models may memorize the training data instead of learning general patterns.

5. Difficult Visualization

- 2 features → Easy to plot
- 3 features → Still possible
- 10,000 features → Impossible to visualize directly

Dimensionality Reduction?

Dimensionality reduction reduces the number of features while preserving as much useful information as possible.

Example:

Original Features = 10,000

↓

Reduced Features = 100

Feature Selection

Removes some original features

Original features remain unchanged

Easier to interpret

Dimensionality Reduction

Creates new features from combinations of existing ones

New transformed features are generated

Harder to interpret

Feature Selection

Example: Keep only top 500 words

Dimensionality Reduction

Example: PCA combines features into principal components

Principal Component Analysis (PCA)

What is PCA?

Principal Component Analysis (PCA) is an **unsupervised** dimensionality reduction technique.

It transforms the original features into a smaller set of **principal components** that preserve the maximum amount of information (variance).

Example:

Student	Height	Weight
A	160	55
B	165	60
C	170	65

Height and weight are strongly related.

Instead of keeping both features separately, PCA can combine them into a new feature called a **principal component**.

Original Features

Height

Weight

Age

↓

PCA

↓

PC1

PC2

PC3

Original Data

*

*

*

*

*

↓

PCA finds the direction where the points spread the most.

↓

Project data onto that direction.

↓

One new feature instead of two.

Advantages

- Reduces dimensions effectively.
- Removes redundancy.
- Speeds up training.

Disadvantages

- New features are harder to interpret.
- Information loss can occur.

Linear Discriminant Analysis (LDA)

Linear Discriminant Analysis is a supervised dimensionality reduction technique.

Unlike PCA, it uses **class labels** to reduce dimensions while maximizing the separation between different classes.

Suppose we have two classes:

Positive Reviews

□

Negative Reviews

Use labeled training data.

- Calculate how samples are distributed within each class.
- Calculate how classes differ from each other.
- Find directions that maximize class separation.
- Project the data onto those directions.

Example

Classify emails as:

- Spam
- Not Spam

Each email has thousands of TF-IDF features.

LDA reduces the feature space while keeping the two classes well separated.

Feature	PCA	LDA
Full Form	Principal Component Analysis	Linear Discriminant Analysis
Learning Type	Unsupervised	Supervised
Uses Class Labels	No	Yes
Goal	Preserve maximum variance	Maximize class separation
Best For	Compression and visualization	Classification tasks
Output	Principal Components	Linear Discriminants

Sentiment Analysis

Sentiment Analysis (also called **Opinion Mining**) is the process of using **Natural Language Processing (NLP)**, **Machine Learning (ML)**, and **Artificial Intelligence (AI)** to determine the emotional tone or opinion expressed in a piece of text.

It identifies whether a sentence expresses:

- Positive sentiment
- Negative sentiment
- Neutral sentiment

Real-Life Examples

Example 1

This mobile phone is amazing.

Sentiment

Positive

Example 2

Battery backup is terrible.

Sentiment

Negative

Example 3

The phone weighs 180 grams.

Sentiment

Neutral

Sentiment Analysis Pipeline

Text

↓

Preprocessing

↓

Feature Extraction

↓

Model

↓

Prediction

↓

Positive / Negative / Neutral

Text Preprocessing

Convert text into a clean format.

Steps

- Lowercase
- Remove punctuation
- Remove URLs
- Remove HTML
- Remove stop words
- Tokenization
- Stemming/Lemmatization

Example

Original

This Phone is AWESOME!!!

↓

phone awesome

Feature Extraction

Convert text into numbers.

Methods

- Bag of Words
- TF-IDF
- Word2Vec
- GloVe

Classification Algorithms

Common ML models

- Naive Bayes
- Logistic Regression
- Decision Tree
- Random Forest
- SVM

```
from textblob import TextBlob
```

```
text = "This phone is amazing."
```

```
blob = TextBlob(text)
```

```
print(blob.sentiment)
```

O/P:

```
Sentiment(
```

```
polarity=0.60,
```

```
subjectivity=0.75
```

```
)
```

Polarity:

Range: -1 to +1

Polarity measures **how positive or negative** the text is.

Value Meaning

+1 Positive

0 Neutral

-1 Negative

Polarity	Meaning
+1	Strongly Positive
+0.60	Positive
0	Neutral
-0.60	Negative
-1	Strongly Negative

Subjectivity:

Range

0 to 1

Subjectivity measures whether the sentence is based on opinion or fact.

Value	Meaning
0	Objective (Fact)
1	Subjective (Opinion)

Value	Meaning
0	Completely Objective (Fact)
0.5	Partially Opinion
1	Completely Subjective (Opinion)

```
from textblob import TextBlob
```

```
text = "This phone is excellent."
```

```
blob = TextBlob(text)
```

```
print("Polarity:", blob.sentiment.polarity)
```

```
print("Subjectivity:", blob.sentiment.subjectivity)
```

```
*****
```

Sentiment Analysis Techniques

1. Lexicon-Based Approach

Uses a dictionary of positive and negative words.

Example Dictionary

Word	Score
------	-------

excellent	+3
-----------	----

amazing	+2
---------	----

bad	-2
-----	----

terrible	-3
----------	----

Sentence

The food is amazing.

Score

+2

Positive

2. Machine Learning Approach

Workflow

Reviews

↓

Preprocessing

↓

TF-IDF

↓

Classifier

↓

Prediction

Algorithms

- Naive Bayes
- Logistic Regression
- SVM
- Random Forest

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
```

```
analyzer = SentimentIntensityAnalyzer()
```

```
text = "This phone is excellent."
```

```
score = analyzer.polarity_scores(text)
```

```
print(score)
```

O/P:{

'neg':0.0,

```
'neu':0.32,
'pos':0.68,
'compound':0.85
}
```

Score	Meaning
pos	Positive score
neg	Negative score
neu	Neutral score
compound	Overall sentiment (-1 to +1)

TextBlob	VADER
General text	Social media text
Polarity & Subjectivity	Detailed sentiment scores
Easy to use	Better for emojis, punctuation, capitalization

Feature	VADER	TextBlob
Full Form	Valence Aware Dictionary and Sentiment Reasoner	No full form (it's a Python NLP library)
Type	Lexicon-based sentiment analyzer	NLP library with sentiment analysis
Best For	Social media posts, tweets, short reviews	General text and simple sentiment tasks
Handles Emojis	Yes	Limited
Handles Negation	Yes	Basic support
Handles Capitalization & Punctuation	Yes	No
Output	Positive, Negative, Neutral, Compound	Polarity and Subjectivity

Valence **A**ware **D**ictionary and **S**entiment **R**easoner

V – Valence

- **Valence** means the **emotional strength or polarity** of a word.
- It indicates whether a word expresses a **positive** or **negative** emotion.

Examples:

Word	Valence
Excellent	Positive
Amazing	Positive
Bad	Negative
Terrible	Negative

A – Aware

VADER is **aware of context** that affects sentiment, such as:

- Capital letters
- Punctuation
- Degree modifiers
- Emojis
- Negations

Example:

This phone is GOOD!!

The capital letters and exclamation marks make the sentiment stronger than:

This phone is good.

D – Dictionary

VADER uses a **predefined sentiment dictionary (lexicon)**.

Example:

Word	Sentiment Score
good	+1.9
excellent	+2.7
amazing	+3.1
bad	-2.5
terrible	-3.1

R – Reasoner

The **Reasoner** applies rules to combine word scores and produce an overall sentiment.

For example:

This phone is not good.

- The word **good** is positive.
- The word **not** changes the meaning.

Practical: VADER

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

analyzer = SentimentIntensityAnalyzer()

text = "This phone is excellent."

score = analyzer.polarity_scores(text)

print(score)
```

Output

```
{
'neg':0.0,
'neu':0.32,
'pos':0.68,
'compound':0.85
}
```

Score	Meaning
pos	Positive score
neg	Negative score
neu	Neutral score
compound	Overall sentiment (-1 to +1)

TextBlob vs VADER

TextBlob	VADER
General text	Social media text
Polarity & Subjectivity	Detailed sentiment scores
Easy to use	Better for emojis, punctuation, capitalization

Machine Learning Example:

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
texts = [
```

```
    "Good phone",
```

```
    "Bad battery",
```

```
    "Excellent camera",
```

```
    "Worst service"
```

```
]
```

```
labels = [
```

```
    "Positive",
```

```
    "Negative",
```

```
    "Positive",
```

```
    "Negative"
```

```
]
```

```
vectorizer = TfidfVectorizer()
```

```
X = vectorizer.fit_transform(texts)
```

```
model = MultinomialNB()
```

```
model.fit(X, labels)
```

```
test = vectorizer.transform(["Excellent battery"])
print(model.predict(test))
*****
```

Flow Diagram:

Training Sentences

|



TF-IDF Vectorizer

|



Numerical Feature Matrix

|



Multinomial Naive Bayes

|



Learns word probabilities for Positive and Negative classes

|



New Sentence

"Excellent battery"

|



TF-IDF Vector

|



Calculate probability for Positive and Negative

|



Choose class with higher probability

|



Prediction
